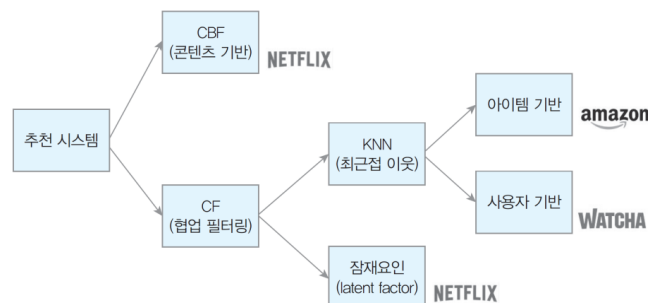


[Chap 15] 추천 시스템



추천 시스템(recommendation system)

- 고객의 취향과 관심에 부합하는 상품과 콘텐츠를 추천함으로써 고객에게 더욱 오래 서비스를 경험하게 하고 추가의 구매를 유도
 - Amazon, 쿠팡, Youtube, Netflix 등 쇼핑물, OTT 등에서 널리 사용
 - 축적된 구매행동, 평가 등의 데이터를 기반으로 고객의 취향을 알아냄
- 추천 시스템의 종류
 - 크게 콘텐츠기반 필터링(CBF, Content Based Filtering)과 협업 필터링(CF, Collaborative Filtering)으로 구분



[그림 15-1] 추천 시스템의 종류

콘텐츠 기반 필터링 (CBF: Content Based Filtering)

- 콘텐츠를 기반으로 사용자가 이전에 구매한 아이템 중에서 유사한 아이템들을 추천
 - 영화의 장르와 감독, 출연진 등 콘텐츠 속성을 벡터로 표현하고, 벡터들 간에 유사도 평가를 통해 높은 유사도의 영화를 추천
 - 다른 사용자의 데이터가 필요하지 않아 새롭거나 대중적이지 않은 아이템에 대한 추천이 가능하고, 사용자에게 추천의 이유를 설명할 수 있는 장점이 있음
 - 데이터가 부족한 새로운 고객에 대해서는 추천이 어렵고, 값으로 유형화하기 어려운 특성치들이 존재하는 경우가 많으며, 추천이 단조로워질 수 있는 단점

〈표 15-1〉 콘텐츠 기반 필터링

구분	영화제목	장르	감독	출연
내가 이전에 본 영화	Arrival	SF, 드라마, 미스터리	드니 빌뇌브	에이미 아담스 제레미 러너
	프로메테우스	SF, 액션, 스릴러	리들리 스콧	노미 라마스 마이클 패스벤더
	마션	SF, 드라마, 코미디	리들리 스콧	맷 데이먼
▼				
추천 영화	블레이드 러너 2049	SF, 액션, 스릴러	리들리 스콧	라이언 고슬링 헤리슨 포트

3

협업 필터링 (CF, Collaborative Filtering)

- 사용자 기반 협업 필터링
 - 나와 비슷한 취향의 사람들을 찾아서 해당 사용자가 높이 평가한 아이템을 추천
 - 사용자 3은 사용자 1과 성향이 비슷함 → 사용자 1은 아이템 E를 높이 평가
 - 사용자 3에게 추천할 아이템은? → E

〈표 15-2〉 사용자 기반 협업 필터링을 위한 평점 행렬

아이템 사용자	아이템 A	아이템 B	아이템 C	아이템 D	아이템 E	아이템 F
사용자 1	4	5	2	1	5	2
사용자 2	1	3	5	4	2	5
사용자 3	5	4	2	2	(데이터 없음)	(데이터 없음)

$$\text{similarity}(1,3) = \frac{(4,5,2,1) \cdot (5,4,2,2)}{|(4,5,2,1)| \cdot |(5,4,2,2)|} = \frac{46}{6.78 \times 7} = 0.97$$

$$\text{similarity}(2,3) = \frac{(1,2,4,4) \cdot (5,4,2,2)}{|(1,2,4,4)| \cdot |(5,4,2,2)|} = \frac{29}{6.08 \times 7} = 0.68$$

4

협업 필터링 (계속)

- 아이템 기반 협업 필터링
 - 평가 패턴이 비슷한 아이템을 추천
 - 사용자 E는 인셉션을 높이 평가함 → 인셉션은 테넷과 더 유사함
 - 사용자 E에게 추천할 아이템은? → 테넷

〈표 15-3〉 아이템 기반 협업 필터링을 위한 평점 행렬

사용자 아이템	사용자 A	사용자 B	사용자 C	사용자 D	사용자 E
인셉션	4	5	3	2	5
테넷	5	5	1	2	(데이터 없음)
미드나잇 인 파리	2	1	4	5	(데이터 없음)

$$\text{similarity}(\text{인셉션}, \text{테넷}) = \frac{(4, 5, 3, 2) \cdot (5, 4, 2, 2)}{|(4, 5, 3, 2)| \cdot |(5, 4, 2, 2)|} = \frac{35}{7.35 \times 7.42} = 0.70$$

$$\text{similarity}(\text{인셉션}, \text{미드나잇 인 파리}) = \frac{(4, 5, 3, 2) \cdot (2, 1, 4, 5)}{|(4, 5, 3, 2)| \cdot |(2, 1, 4, 5)|} = \frac{29}{7.35 \times 6.78} = 0.58$$

5

협업 필터링 (계속)

- 잠재요인 협업 필터링 (latent factor collaborative filtering)
 - 행렬 분해(matrix factorization)를 이용
 - Singular Value Decomposition, SVD: $A = U\Sigma V^T$, (여기서 $A: (m \times n), \Sigma: (m \times n), V: (n \times n)$)
 - Truncated SVD
 - 평점 매트릭스는 매우 희소행렬이므로 차원 축소가 가능

$$A \simeq U_k \Sigma_k V_k^T = PQ^T \text{ (here } P: (m \times k), Q: (n \times k))$$
 - 메모리 양이 기존의 $(m \times n)$ 에서 $(m \times k) + (n \times k)$ 로 감소

예) 만약 사용자 100만명($m=100$ 만), 아이템 1만개($n=1$ 만)이면 원래 평점 행렬에는 100억개의 값이 있으나, $k=100$ 으로 둔다면 분해된 행렬은 $(100\text{만} \times 100) + (1\text{만} \times 100) = 1100\text{만}$ 개로 크기가 크게 감소
 - 여기서 P, Q는 유일하지는 않음
 - 단점
 - 잠재요인의 정확한 의미를 알 수 없는 경우가 많아 사용자에게 추천이 된 이유를 설명하기가 어려움
 - 새로운 값들이 추가되면 이를 반영하는 것이 어려움

6

협업 필터링 (계속)

- 잠재요인 협업 필터링의 예시
 - 주어진 사용자-아이템 행렬

〈표 15-4〉 잠재요인 협업 필터링을 위한 평점 행렬

	에일리언	블레이드러너	프로메테우스	미드나잇 인 파리	라라랜드
사용자 1	4	5	4	1	2
사용자 2		5	5		1
사용자 3	1			5	4
사용자 4	1	1	2	5	

- P, Q 행렬로 분해

	에일리언	블레이드러너	프로메테우스	미드나잇 인 파리	라라랜드
사용자 1	4	5	4	1	2
사용자 2		5	5		1
사용자 3	1			5	4
사용자 4	1	1	2	5	

 $=$

	요인 1	요인 2
사용자 1	1.63	-1.49
사용자 2	1.59	-1.77
사용자 3	1.83	1.08
사용자 4	1.93	0.91

 $\cdot$

	에일리언	블레이드러너	프로메테우스	미드나잇 인 파리	라라랜드
요인 1	1.22	1.32	1.56	1.96	1.73
요인 2	-1.28	-1.74	-1.23	1.31	0.81

- P, Q의 곱은 원래 행렬을 근사적으로 복원
 - 데이터 크기 감소
 - 결측치에 대한 추정치 제공
 - 사용자 2에게 추천할 아이템은? → 에일리언

〈그림 15-2〉 특이값 분해 결과

	요인 1	요인 2
사용자 1	1.63	-1.49
사용자 2	1.59	-1.77
사용자 3	1.83	1.08
사용자 4	1.93	0.91

 $\cdot$

	에일리언	블레이드러너	프로메테우스	미드나잇 인 파리	라라랜드
요인 1	1.22	1.32	1.56	1.96	1.73
요인 2	-1.28	-1.74	-1.23	1.31	0.81

 $=$

	에일리언	블레이드러너	프로메테우스	미드나잇 인 파리	라라랜드
사용자 1	3.9	4.8	4.4	1.2	1.6
사용자 2	4.2	5.2	4.7	0.8	1.3
사용자 3	0.9	0.5	1.5	5.0	4.0
사용자 4	1.2	1.0	1.9	5.0	4.1

〈그림 15-3〉 근사값 복원과 결측치 추정

7

콘텐츠 기반 필터링 추천 시스템(CBF) 구현

- TMDB(The Movie Database) 영화 데이터 활용
 - 총 4800여개의 영화에 대해 투자된 예산, 장르, 키워드, 언어, 제목 등 영화에 대한 자세한 정보가 정리
- 장르만 가지고 영화를 추천해보고자 함

〈표 15-5〉 TMDB 데이터셋의 제목과 장르 열

original_title	genres
Avatar	[{"id": 28, "name": "Action"}, {"id": 12, "name": "Adventure"}, {"id": 14, "name": "Fantasy"}, {"id": 878, "name": "Science Fiction"}]
Pirates of the Caribbean: At World's End	[{"id": 12, "name": "Adventure"}, {"id": 14, "name": "Fantasy"}, {"id": 28, "name": "Action"}]
Spectre	[{"id": 28, "name": "Action"}, {"id": 12, "name": "Adventure"}, {"id": 80, "name": "Crime"}]
The Dark Knight Rises	[{"id": 28, "name": "Action"}, {"id": 80, "name": "Crime"}, {"id": 18, "name": "Drama"}, {"id": 53, "name": "Thriller"}]
John Carter	[{"id": 28, "name": "Action"}, {"id": 12, "name": "Adventure"}, {"id": 878, "name": "Science Fiction"}]
Spider-Man 3	[{"id": 14, "name": "Fantasy"}, {"id": 28, "name": "Action"}, {"id": 12, "name": "Adventure"}]
Tangled	[{"id": 16, "name": "Animation"}, {"id": 10751, "name": "Family"}]
Avengers: Age of Ultron	[{"id": 28, "name": "Action"}, {"id": 12, "name": "Adventure"}, {"id": 878, "name": "Science Fiction"}]
Harry Potter and the Half-Blood Prince	[{"id": 12, "name": "Adventure"}, {"id": 14, "name": "Fantasy"}, {"id": 10751, "name": "Family"}]

8

데이터 전처리

- 데이터를 읽어들이고 제목과 장르만 뽑아서 별도의 데이터프레임으로 만들기

```
1 import pandas as pd
2 movie_full_data = pd.read_csv('tmdb_5000_movies.csv')
3 movie = movie_full_data[['original_title', 'genres']]
```

- 각 영화별로 장르의 이름만 뽑아서 문자열 리스트로 만들기

```
1 from ast import literal_eval
2 for i in range(len(movie)):
3     mv = movie.iloc[i]
4     mv['genres'] = literal_eval(mv['genres'])
5     mv['genres'] = [y['name'] for y in mv['genres']]
6 print(movie)
```

	original_title	genres
0	Avatar	[Action, Adventure, Fantasy, Science Fiction]
1	Pirates of the Caribbean: At World's End	[Adventure, Fantasy, Action]

9

데이터 전처리 (계속)

- 장르를 숫자 벡터로 변환
 - 하나의 영화가 여러 장르에 해당하므로, 장르 개수 크기의 벡터로 만들어 해당 열을 값을 1로 함

〈표 15-6〉 장르의 벡터화

제목	Action	Adventure	Fantasy	Science Fiction	Crime	Drama	Thriller	Comedy	Romance	...
Avatar	1	1	1	1	0	0	0	0	0	...
Pirates of the Ca..	1	1	1	0	0	0	0	0	0	...
Spectre	1	1	0	0	1	0	0	0	0	...
The Dark Knight..	1	0	0	0	1	1	1	0	0	...
John Carter	1	1	0	1	0	0	0	0	0	...

데이터 전처리 (계속)

- 장르를 숫자 벡터로 변환 (계속)
 - 존재하는 모든 장르를 나열 : 총 20가지의 장르가 있음

```
1 all_genres = []
2 for i in range(len(movie)):
3     mv = movie.iloc[i]
4     for g in mv['genres']:
5         if g not in all_genres: all_genres.append(g)
6 print(all_genres)
```

```
['Action', 'Adventure', 'Fantasy', 'Science Fiction', 'Crime', 'Drama', 'Thriller',
'Animation', 'Family', 'Western', 'Comedy', 'Romance', 'Horror', 'Mystery', 'History',
'War', 'Music', 'Documentary', 'Foreign', 'TV Movie']
```

- (1행) all_genres라는 리스트를 초기화
- (3행) 각 행의 영화를 mv에 넣음
- (4~5행) 각 영화별 장르 리스트인 mv['genres']의 각 항목 g에 대해 만약 g가 all_genres에 없으면 추가

11

데이터 전처리 (계속)

- 장르를 숫자 벡터로 변환 (계속)
 - 각 영화별로 장르를 벡터화하여 gvec 열에 저장

```
1 import numpy as np
2 for i in range(len(movie)):
3     mv = movie.iloc[i]
4     vec = []
5     for g in all_genres:
6         if g in mv['genres']: vec.append(1)
7         else: vec.append(0)
8     movie.loc[i, 'gvec'] = np.asarray(vec, dtype='float32')
9 print(movie)
```

	original_title	...	gvec
0	Avatar	...	[1.0, 1.0, 1.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, ...]
1	Pirates of the Caribbean: At World's End	...	[1.0, 1.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ...]
2	Spectre	...	[1.0, 1.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, ...]
3	The Dark Knight Rises	...	[1.0, 0.0, 0.0, 0.0, 1.0, 1.0, 1.0, 0.0, 0.0, ...]
4	John Carter	...	[1.0, 1.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, ...]
...	...	...	...

- (3행) 각 영화 데이터 행을 mv에 넣고, 3행에서 vec이라는 이름의 빈 리스트
- (5행) all_genres의 각 장르명을 g에 넣음
- (6~7행) 만약 g가 해당 영화 장르명 리스트인 mv['genres']에 있으면 1을, 아니면 0을 차례로 vec 리스트에 붙여나감
- (8행) 이렇게 만들어진 vec 리스트를 numpy 배열로 바꾸어 movie 데이터프레임의 해당 위치에 저장

12

유사도 계산 함수

- 14장의 코사인 유사도 함수 참고

```
1 def magnitude(v):
2     m=0
3     for x in v: m += x**2
4     return (np.sqrt(m))
5
6 def similarity(a,b):
7     dot = np.dot(a,b)
8     if dot==0: return 0
9     ma = magnitude(a)
10    mb = magnitude(b)
11    return(dot/(ma*mb))
```

- (8행) similarity 함수 내용에서 'if dot==0: return 0' 추가
 - 데이터 중에 간혹 장르가 기입되지 않은 데이터가 있어서, 분모의 절대값이 0이 되어 계산 오류가 발생할 수 있기 때문에 내적이 0이면 바로 유사도를 0으로 리턴

13

사용자 입력 및 추천 출력

- 키보드로 영화 번호를 입력 받아서 해당 영화의 제목을 보여줌

```
1 while True:
2     movie_num_str = input("\nEnter movie number: ")
3     if movie_num_str == '': break
4     movie_num = int(movie_num_str)
5     print("You selected: ", movie.loc[movie_num, 'original_title'])
```

- (2행) movie_num_str에 영화 번호를 입력받음
- (4행) 영화 번호를 숫자값으로 바꾸어 movie_num에 저장
- (5행) 입력받은 영화의 제목을 출력
- 입력한 영화와 다른 모든 영화의 유사도 값을 계산하여 sim_list에 추가

```
1 my_vec = movie.loc[movie_num, 'gvec']
2 sim_list = []
3 for i in range(len(movie)):
4     if i==movie_num: sim_list.append(0)
5     else:
6         mv = movie.iloc[i]
7         sim = similarity(my_vec, mv['gvec'])
8         sim_list.append(sim)
```

14

사용자 입력 및 추천 출력 (계속)

- 유사도 벡터에서 가장 높은 값 10개를 찾아서, 해당 영화의 제목을 출력

```
1 y = np.argsort(sim_list)[::-1]
2 print("Recommended:")
3 for i in y[0:10]:
4     print('%4d%i, '%6.4f'%sim_list[i],movie.loc[i,'original_title'])
```

- (1행) sim_list를 내림차순으로 정렬한 인덱스의 값들을 리스트로 만들어 y에 저장
 - sim_list의 값이 [0.8, 0.2, 0.3]이었다면 이것을 오름차순으로 정렬하면 [0.2, 0.3, 0.8]일 것이고, np.argsort는 이 값들의 인덱스 순서는 [1, 2, 0]을 계산해줌
 - [: : -1]은 리스트를 역순으로 만들어 내림차순 인덱스가 되게 함
- (3~4행) y의 처음부터 10개까지의 값을 i에 넣어서 해당 유사도값 sim_list[i]와 해당 번호의 영화제목 movie.loc[i, 'original_title']을 출력

전체 코드: ex15-1.py

15

사용자 입력 및 추천 출력 (계속)

- 출력 결과
 - 임의의 영화 번호를 입력하면 유사 장르의 영화를 잘 추천함을 확인

```
Enter movie number: 0
You selected: Avatar
Recommended:
 61 1.0000 Jupiter Ascending
232 1.0000 The Wolverine
870 1.0000 Superman II
 10 1.0000 Superman Returns
813 1.0000 Superman
3494 1.0000 Beastmaster 2: Through the Portal of Time
 14 1.0000 Man of Steel
 46 1.0000 X-Men: Days of Future Past
3208 0.8944 Star Wars: Clone Wars (Volume 1)
1390 0.8944 Underworld: Rise of the Lycans

Enter movie number: 191
You selected: Harry Potter and the Prisoner of Azkaban
Recommended:
197 1.0000 Harry Potter and the Philosopher's Stone
341 1.0000 Percy Jackson & the Olympians: The Lightning Thief
781 1.0000 Inkheart
 37 1.0000 Oz: The Great and Powerful
143 1.0000 Pan
114 1.0000 Harry Potter and the Goblet of Fire
109 1.0000 The Chronicles of Narnia: The Voyage of the Dawn Treader
368 1.0000 Percy Jackson: Sea of Monsters
276 1.0000 Harry Potter and the Chamber of Secrets
 63 1.0000 The Chronicles of Narnia: The Lion, the Witch and the Wardrobe
```

16